
JBlackJack

Technical Project Report

Francesco Zompanti

Student ID: 2012601

Course: Attendance M–Z

Software version: 2.0.0-SNAPSHOT

Technologies: Java 17, Swing, Maven, JUnit 5

Report date: August 7, 2026

Contents

1	Introduction	1
1.1	Main features	1
2	Requirements and Game Rules	2
2.1	Functional requirements	2
2.2	Implemented rules	2
3	Software Architecture	3
3.1	Package structure	3
3.2	Model–View–Controller	3
3.3	UML diagram	3
4	Domain Model	5
4.1	Cards and factory	5
4.2	Deck	5
4.3	Players and Strategy	5
4.4	Game state and events	6
5	User Interface and Control Flow	7
5.1	Startup and navigation	7
5.2	Game table	7
5.3	Controller and timing	7
5.4	Optional audio	7
6	Profile Persistence	8
7	Build, Execution, and Testing	9
7.1	Application build	9
7.2	Automated tests	9
7.3	Building this report	9
8	Design Assessment and Future Work	10
	Conclusion	11

Chapter 1

Introduction

JBlackJack is a desktop application that simulates a Blackjack game between one human player, two automated opponents, and the dealer. It is written in Java 17 and uses Swing for its graphical user interface. In addition to the game rules, it provides local player profiles, persistent statistics, a progression system, and optional image and audio resources.

The main design goal is a clear separation between rules, presentation, and coordination. The code follows Model–View–Controller (MVC) and also applies the Strategy, Factory Method, Observer/Listener, and Singleton patterns. A Maven build and JUnit tests make the implementation reproducible and verifiable.

1.1 Main features

- a standard 52-card deck and the core rules of Blackjack;
- *Hit* and *Stand* actions, automated opponents, and dealer turns;
- correct handling of aces as either 1 or 11 to prevent a bust;
- a hidden dealer hole card until the end of the round;
- validated local profiles with statistics and experience points;
- a responsive Swing interface updated on the Event Dispatch Thread;
- graceful fallback when optional images or audio files are unavailable;
- automated tests for domain rules and profile persistence.

Chapter 2

Requirements and Game Rules

2.1 Functional requirements

ID	Requirement
FR-01	The user can create or load a profile using a nickname.
FR-02	The system deals two initial cards to every participant and the dealer.
FR-03	During their turn, users can hit or stand.
FR-04	Bots and the dealer choose their actions through replaceable strategies.
FR-05	At the end of a round, the system determines the result against the dealer.
FR-06	Played, won, and lost games and experience points are stored persistently.
FR-07	Users can review their statistics and start another round.

2.2 Implemented rules

Number cards have their rank value, face cards are worth 10, and an ace initially counts as 11. If a total exceeds 21, each ace can be reduced from 11 to 1 by subtracting 10 until the hand is valid or no aces remain. A value above 21 is a bust. The dealer and bots hit below 17 and stand on 17 or more. The user wins by holding a valid hand higher than the dealer's, or when the dealer busts. A tie changes neither wins nor losses but still counts as a played game.

Chapter 3

Software Architecture

3.1 Package structure

Package	Responsibility
<code>main.blackjack</code>	application entry point and GUI startup on the EDT;
<code>main.model</code>	cards, deck, players, strategies, game state, profiles, and events;
<code>main.controller</code>	translation of GUI input into model operations;
<code>main.view</code>	Swing windows for profiles, menus, statistics, and the game table;
<code>main.util</code>	optional cross-cutting services, particularly audio playback.

3.2 Model–View–Controller

The **Model** owns state and enforces rules without depending on Swing. The **View** presents that state and exposes interaction controls. The **Controller** connects buttons to model operations, schedules automated turns, and activates audio feedback.

The model emits typed events through `GameListener`. The game view registers as a listener and reacts to `STATE_CHANGED` and `GAME_OVER`. Consequently, the model has no knowledge of Swing components and can be tested independently.

3.3 UML diagram

Figure 3.1 provides an overview of the main classes and relationships. The diagram is maintained as a separate project artifact in the `UML/` directory and is included automatically when this report is built.



Figure 3.1: JBlackJack UML class diagram.

Chapter 4

Domain Model

4.1 Cards and factory

The `Card` interface defines a card's value, suit, and rank. Its default `getImageFileName()` method establishes the naming convention for image assets. The concrete implementations are `NumericCard` for ranks 2 through 10, `FaceCard` for jack, queen, and king, and `AceCard` for aces.

`CardFactory.createCard(suit,rank)` centralizes construction and validates both arguments. Unknown suits, out-of-range numbers, and invalid ranks cause an `IllegalArgumentException`, preventing inconsistent cards from entering a deck.

4.2 Deck

`Deck` generates the Cartesian product of four suits and thirteen ranks with Streams, producing 52 cards. Each game owns an independent deck instance. The `reset()` method rebuilds and shuffles it, while `drawCard()` removes one card and recreates the deck if it is empty. Avoiding global deck state reduces coupling between games and simplifies testing.

4.3 Players and Strategy

`Player` aggregates a name, hand, human-player flag, and `PlayerStrategy`. The hand accessor returns an unmodifiable view, so cards can only be changed through the domain API. Hand scoring sums all values and progressively reduces aces while the total exceeds 21.

The decision to hit is delegated to `PlayerStrategy` implementations: `BotStrategy`, `DealerStrategy`, and `HumanPlayerStrategy`. Human input is handled by the controller, so the human strategy makes no automatic choice.

Listing 4.1: Core algorithm for scoring flexible aces.

```
1 int value = hand.stream().mapToInt(Card::getValue).sum();
2 long aces = hand.stream()
3     .filter(card -> "ace".equals(card.getRank()))
4     .count();
5 while (value > 21 && aces-- > 0) {
6     value -= 10;
7 }
8 return value;
```

4.4 Game state and events

`BlackjackModel` owns three players, the dealer, deck, user profile, and current turn index. A round resets and shuffles the deck, deals two cards to every participant, runs the human and automated turns, plays the dealer's hand, evaluates the result, updates and saves the profile, and finally emits `GAME_OVER`.

Collections exposed by the model are unmodifiable. Listeners are stored in a `CopyOnWriteArrayList`, allowing safe iteration even if registrations change during event delivery.

Chapter 5

User Interface and Control Flow

5.1 Startup and navigation

`JBlackJack.main()` calls `SwingUtilities.invokeLater`, ensuring that all component creation takes place on the Event Dispatch Thread. Navigation begins with `ProfileCreationView`, continues to `MainMenuView`, and then opens either `StatisticsView` or a game in `BlackjackView`.

Profile nicknames contain between 1 and 24 Unicode letters, digits, hyphens, or underscores. This improves both usability and the safety of filenames used for local storage.

5.2 Game table

`BlackjackView` implements `GameListener`. On each event it refreshes the dealer and player areas, changes the status text, and enables controls only during the human turn. The dealer's second card remains hidden until the round is complete.

Images are loaded from the classpath. If an expected card or avatar asset is missing, the GUI uses a textual fallback and remains fully usable.

5.3 Controller and timing

`BlackjackController` registers the buttons' action listeners. After *Hit*, it checks for a bust; after *Stand*, it advances to the next player. Bot turns are scheduled with a `javax.swing.Timer`, keeping all updates on the EDT without blocking the window. Once all players finish, the controller asks the model to play the dealer and close the round.

5.4 Optional audio

`AudioManager` is a stateless Singleton that loads WAV clips from the classpath through the public `javax.sound.sampled` API. It closes clips after playback and handles missing or invalid resources without interrupting the game. Audio feedback is therefore optional rather than an application dependency.

Chapter 6

Profile Persistence

`UserProfile` stores the nickname, avatar, games played, wins, losses, level, and experience. Each win awards 100 points. The first level requires 1,000 points and the threshold grows with the current level.

Data is written to `profiles/<nickname>_profilo.txt`. Saving first creates a temporary file and then uses an atomic move when supported by the filesystem. This prevents a partially written profile after an interruption. Loading validates the field count and numeric formats; missing or malformed data leaves default values unchanged and returns a failure result that the caller can handle.

Nickname validation blocks path components such as `../`, preventing the constructed path from escaping the profile directory. Tests can inject a temporary directory into the profile constructor and remain isolated from real user data.

Chapter 7

Build, Execution, and Testing

7.1 Application build

The application requires JDK 17 or newer and Maven 3.9 or newer. From the repository root, it can be tested and started with:

```
mvn clean test
mvn exec:java
```

The recommended pre-release command is `mvnverify`. The `pom.xml` file configures the historical `src/main` source directory, Java 17 compilation, JUnit Jupiter, and the execution plugin with `main.blackjack.JBlackJack` as main class.

7.2 Automated tests

Test class	Verified behavior
<code>CardFactoryTest</code>	valid construction and rejection of invalid suits or ranks;
<code>PlayerTest</code>	single and multiple aces and immutability of the exposed hand;
<code>StrategyTest</code>	dealer decisions for soft hands containing an ace;
<code>UserProfileTest</code>	round-trip storage, malformed input, safe paths, and levels.

Persistence tests use `@TempDir`, so each run receives an empty directory and does not modify `profiles/`. The repository's GitHub Actions workflow can run the same verification in a clean environment.

7.3 Building this report

This report has a dedicated source file and is independent from the Java build. A LaTeX distribution containing `pdflatex` is sufficient:

```
make -C report
```

The generated document is `report/build/JBlackJack-report.pdf`. The `clean` target removes auxiliary files, while `distclean` also removes the PDF. Running `pdflatex` twice resolves the table of contents and cross-references.

Chapter 8

Design Assessment and Future Work

The central design choices are MVC separation, encapsulated collections, typed events, and graceful handling of optional assets. Factory Method makes card validation explicit; Strategy isolates game policies; atomic persistence protects local data; correct EDT use keeps Swing responsive; and Maven with JUnit provides a repeatable regression baseline.

Possible extensions include betting and chips, split and double-down actions, configurable bot strategies, controller-level automated tests, and versioned structured profile storage. Generating the UML diagram directly from source during continuous integration would also keep it synchronized automatically.

Conclusion

JBlackJack implements a complete desktop card game with a clear and modular structure. The core rules remain independent from presentation, automated behaviors are replaceable, and statistics persist between sessions. Validation, unmodifiable views, atomic writes, correct Event Dispatch Thread usage, and JUnit tests provide a robust foundation for future development without sacrificing the simplicity of the current implementation.